

SC1001: Introduction to Computational Thinking

Week 5: Strings and Data Structures

Jordan Boyd-Graber

Nanyang Technological University

Slides

Slides and updates will be posted at

boydgraber.org/teaching/SC_1001/

Overview

1. Predict the output of common string operations
2. Trace how list methods change a list
3. Read slices and indexing from a list of lists
4. Build list comprehensions step by step
5. Use a dictionary as a small book database
6. Find palindromes in a sentence
7. Translate text into B-Sprache

Discussion 1: What Is the Output?

```
sample_string = "Hello, World! Welcome to Python programming."  
upper_string = sample_string.upper()  
print(f"Uppercase: {upper_string}")  
lower_string = sample_string.lower()  
print(f"Lowercase: {lower_string}")  
capitalized_string = sample_string.capitalize()  
print(f"Capitalized: {capitalized_string}")  
position = sample_string.find("World")  
print(f"Position of 'World': {position}")  
replaced_string = sample_string.replace("World", "Universe")  
print(f"Replaced string: {replaced_string}")  
words_list = sample_string.split()  
print(f"List of words: {words_list}")  
joined_string = " ".join(words_list)  
print(f"Joined string: {joined_string}")
```

Discussion 1: What Is the Output?

```
sample_string = "Hello, World! Welcome to Python programming."
upper_string = sample_string.upper()
print(f"Uppercase: {upper_string}")
lower_string = sample_string.lower()
print(f"Lowercase: {lower_string}")
capitalized_string = sample_string.capitalize()
print(f"Capitalized: {capitalized_string}")
position = sample_string.find("World")
print(f"Position of 'World': {position}")
replaced_string = sample_string.replace("World", "Universe")
print(f"Replaced string: {replaced_string}")
words_list = sample_string.split()
print(f"List of words: {words_list}")
joined_string = " ".join(words_list)
print(f"Joined string: {joined_string}")
```

Work down the prints one at a time.

Discussion 1: What Is the Output?

```
sample_string = "Hello, World! Welcome to Python programming."  
upper_string = sample_string.upper()  
print(f"Uppercase: {upper_string}")  
lower_string = sample_string.lower()  
print(f"Lowercase: {lower_string}")  
capitalized_string = sample_string.capitalize()  
print(f"Capitalized: {capitalized_string}")  
position = sample_string.find("World")  
print(f"Position of 'World': {position}")  
replaced_string = sample_string.replace("World", "Universe")  
print(f"Replaced string: {replaced_string}")  
words_list = sample_string.split()  
print(f"List of words: {words_list}")  
joined_string = " ".join(words_list)  
print(f"Joined string: {joined_string}")
```

Uppercase: HELLO, WORLD! WELCOME TO PYTHON PROGRAMMING.

Discussion 1: What Is the Output?

```
sample_string = "Hello, World! Welcome to Python programming."  
upper_string = sample_string.upper()  
print(f"Uppercase: {upper_string}")  
lower_string = sample_string.lower()  
print(f"Lowercase: {lower_string}")  
capitalized_string = sample_string.capitalize()  
print(f"Capitalized: {capitalized_string}")  
position = sample_string.find("World")  
print(f"Position of 'World': {position}")  
replaced_string = sample_string.replace("World", "Universe")  
print(f"Replaced string: {replaced_string}")  
words_list = sample_string.split()  
print(f"List of words: {words_list}")  
joined_string = " ".join(words_list)  
print(f"Joined string: {joined_string}")
```

Lowercase: hello, world! welcome to python programming.

Discussion 1: What Is the Output?

```
sample_string = "Hello, World! Welcome to Python programming."
upper_string = sample_string.upper()
print(f"Uppercase: {upper_string}")
lower_string = sample_string.lower()
print(f"Lowercase: {lower_string}")
capitalized_string = sample_string.capitalize()
print(f"Capitalized: {capitalized_string}")
position = sample_string.find("World")
print(f"Position of 'World': {position}")
replaced_string = sample_string.replace("World", "Universe")
print(f"Replaced string: {replaced_string}")
words_list = sample_string.split()
print(f"List of words: {words_list}")
joined_string = " ".join(words_list)
print(f"Joined string: {joined_string}")
```

Capitalized: Hello, world! welcome to python programming.

Discussion 1: What Is the Output?

```
sample_string = "Hello, World! Welcome to Python programming."  
upper_string = sample_string.upper()  
print(f"Uppercase: {upper_string}")  
lower_string = sample_string.lower()  
print(f"Lowercase: {lower_string}")  
capitalized_string = sample_string.capitalize()  
print(f"Capitalized: {capitalized_string}")  
position = sample_string.find("World")  
print(f"Position of 'World': {position}")  
replaced_string = sample_string.replace("World", "Universe")  
print(f"Replaced string: {replaced_string}")  
words_list = sample_string.split()  
print(f"List of words: {words_list}")  
joined_string = " ".join(words_list)  
print(f"Joined string: {joined_string}")
```

Position of 'World': 7

Discussion 1: What Is the Output?

```
sample_string = "Hello, World! Welcome to Python programming."  
upper_string = sample_string.upper()  
print(f"Uppercase: {upper_string}")  
lower_string = sample_string.lower()  
print(f"Lowercase: {lower_string}")  
capitalized_string = sample_string.capitalize()  
print(f"Capitalized: {capitalized_string}")  
position = sample_string.find("World")  
print(f"Position of 'World': {position}")  
replaced_string = sample_string.replace("World", "Universe")  
print(f"Replaced string: {replaced_string}")  
words_list = sample_string.split()  
print(f"List of words: {words_list}")  
joined_string = " ".join(words_list)  
print(f"Joined string: {joined_string}")
```

Replaced string: Hello, Universe! Welcome to Python programming.

Discussion 1: What Is the Output?

```
sample_string = "Hello, World! Welcome to Python programming."  
upper_string = sample_string.upper()  
print(f"Uppercase: {upper_string}")  
lower_string = sample_string.lower()  
print(f"Lowercase: {lower_string}")  
capitalized_string = sample_string.capitalize()  
print(f"Capitalized: {capitalized_string}")  
position = sample_string.find("World")  
print(f"Position of 'World': {position}")  
replaced_string = sample_string.replace("World", "Universe")  
print(f"Replaced string: {replaced_string}")  
words_list = sample_string.split()  
print(f"List of words: {words_list}")  
joined_string = " ".join(words_list)  
print(f"Joined string: {joined_string}")
```

List of words: ['Hello,', 'World!', 'Welcome', 'to', 'Python', 'programming.']

Discussion 1: What Is the Output?

```
sample_string = "Hello, World! Welcome to Python programming."
upper_string = sample_string.upper()
print(f"Uppercase: {upper_string}")
lower_string = sample_string.lower()
print(f"Lowercase: {lower_string}")
capitalized_string = sample_string.capitalize()
print(f"Capitalized: {capitalized_string}")
position = sample_string.find("World")
print(f"Position of 'World': {position}")
replaced_string = sample_string.replace("World", "Universe")
print(f"Replaced string: {replaced_string}")
words_list = sample_string.split()
print(f"List of words: {words_list}")
joined_string = " ".join(words_list)
print(f"Joined string: {joined_string}")
```

Joined string: Hello, World! Welcome to Python programming.

Discussion 2: What Is the Output?

```
temps = [29, 26, 31, 28, 28, 30, 30]
print("start", temps)
temps.sort()
print("sort ", temps)
new_temp = 27
temps.append(new_temp)
print("add  ", temps)
print("max  ", max(temps))
print("min  ", min(temps))
pre = [28, 24, 27]
pre.append(temps)
print("pre  ", pre)
aft = [31, 32, 30]
for temp in aft:
    temps.append(temp)
print("week2", temps)
print("avg  ", sum(temps) / len(temps))
```

Output

Predict before running.

Discussion 2: What Is the Output?

```
temps = [29, 26, 31, 28, 28, 30, 30]
print("start", temps)
temps.sort()
print("sort ", temps)
new_temp = 27
temps.append(new_temp)
print("add  ", temps)
print("max  ", max(temps))
print("min  ", min(temps))
pre = [28, 24, 27]
pre.append(temps)
print("pre  ", pre)
aft = [31, 32, 30]
for temp in aft:
    temps.append(temp)
print("week2", temps)
print("avg  ", sum(temps) / len(temps))
```

Output

Work through line by line.

Discussion 2: What Is the Output?

```
temps = [29, 26, 31, 28, 28, 30, 30]
print("start", temps)
temps.sort()
print("sort ", temps)
new_temp = 27
temps.append(new_temp)
print("add  ", temps)
print("max  ", max(temps))
print("min  ", min(temps))
pre = [28, 24, 27]
pre.append(temps)
print("pre  ", pre)
aft = [31, 32, 30]
for temp in aft:
    temps.append(temp)
print("week2", temps)
print("avg  ", sum(temps) / len(temps))
```

Output

start [29, 26, 31, 28, 28, 30, 30]

Start with the original list exactly as written.

Discussion 2: What Is the Output?

```
temps = [29, 26, 31, 28, 28, 30, 30]
print("start", temps)
temps.sort()
print("sort ", temps)
new_temp = 27
temps.append(new_temp)
print("add  ", temps)
print("max  ", max(temps))
print("min  ", min(temps))
pre = [28, 24, 27]
pre.append(temps)
print("pre  ", pre)
aft = [31, 32, 30]
for temp in aft:
    temps.append(temp)
print("week2", temps)
print("avg  ", sum(temps) / len(temps))
```

Output

```
start [29, 26, 31, 28, 28, 30,
30]
sort [26, 28, 28, 29, 30, 30, 31]
```

sort() changes temps in place.

Discussion 2: What Is the Output?

```
temps = [29, 26, 31, 28, 28, 30, 30]
print("start", temps)
temps.sort()
print("sort ", temps)
new_temp = 27
temps.append(new_temp)
print("add  ", temps)
print("max  ", max(temps))
print("min  ", min(temps))
pre = [28, 24, 27]
pre.append(temps)
print("pre  ", pre)
aft = [31, 32, 30]
for temp in aft:
    temps.append(temp)
print("week2", temps)
print("avg  ", sum(temps) / len(temps))
```

Output

```
start [29, 26, 31, 28, 28, 30,
30]
sort [26, 28, 28, 29, 30, 30, 31]
add [26, 28, 28, 29, 30, 30, 31,
27]
```

append(new_temp) adds one new number at the end.

Discussion 2: What Is the Output?

```
temps = [29, 26, 31, 28, 28, 30, 30]
print("start", temps)
temps.sort()
print("sort ", temps)
new_temp = 27
temps.append(new_temp)
print("add  ", temps)
print("max  ", max(temps))
print("min  ", min(temps))
pre = [28, 24, 27]
pre.append(temps)
print("pre  ", pre)
aft = [31, 32, 30]
for temp in aft:
    temps.append(temp)
print("week2", temps)
print("avg  ", sum(temps) / len(temps))
```

Output

```
start [29, 26, 31, 28, 28, 30,
30]
sort [26, 28, 28, 29, 30, 30, 31]
add [26, 28, 28, 29, 30, 30, 31,
27]
max 31
```

Now the largest value is 75.

Discussion 2: What Is the Output?

```
temps = [29, 26, 31, 28, 28, 30, 30]
print("start", temps)
temps.sort()
print("sort ", temps)
new_temp = 27
temps.append(new_temp)
print("add  ", temps)
print("max  ", max(temps))
print("min  ", min(temps))
pre = [28, 24, 27]
pre.append(temps)
print("pre  ", pre)
aft = [31, 32, 30]
for temp in aft:
    temps.append(temp)
print("week2", temps)
print("avg  ", sum(temps) / len(temps))
```

Output

```
start [29, 26, 31, 28, 28, 30, 30]
sort [26, 28, 28, 29, 30, 30, 31]
add [26, 28, 28, 29, 30, 30, 31, 27]
max 31
min 26
```

The smallest value stays 68.

Discussion 2: What Is the Output?

```
temps = [29, 26, 31, 28, 28, 30, 30]
print("start", temps)
temps.sort()
print("sort ", temps)
new_temp = 27
temps.append(new_temp)
print("add  ", temps)
print("max  ", max(temps))
print("min  ", min(temps))
pre = [28, 24, 27]
pre.append(temps)
print("pre  ", pre)
aft = [31, 32, 30]
for temp in aft:
    temps.append(temp)
print("week2", temps)
print("avg  ", sum(temps) / len(temps))
```

Output

```
start [29, 26, 31, 28, 28, 30,
30]
sort [26, 28, 28, 29, 30, 30, 31]
add [26, 28, 28, 29, 30, 30, 31,
27]
max 31
min 26
pre [28, 24, 27, [26, 28, 28, 29,
30, 30, 31, 27]]
```

`pre.append(temps)` adds the whole list as one element, so `pre` becomes a nested list.

Discussion 2: What Is the Output?

```
temps = [29, 26, 31, 28, 28, 30, 30]
print("start", temps)
temps.sort()
print("sort ", temps)
new_temp = 27
temps.append(new_temp)
print("add  ", temps)
print("max  ", max(temps))
print("min  ", min(temps))
pre = [28, 24, 27]
pre.append(temps)
print("pre  ", pre)
aft = [31, 32, 30]
for temp in aft:
    temps.append(temp)
print("week2", temps)
print("avg  ", sum(temps) / len(temps))
```

Output

```
start [29, 26, 31, 28, 28, 30, 30]
sort [26, 28, 28, 29, 30, 30, 31]
add [26, 28, 28, 29, 30, 30, 31, 27]
max 31
min 26
pre [28, 24, 27, [26, 28, 28, 29, 30, 30, 31, 27]]
week2 [26, 28, 28, 29, 30, 30, 31, 27]
```

The loop adds the values in aft one at a time to temps.

Discussion 2: What Is the Output?

```
temps = [29, 26, 31, 28, 28, 30, 30]
print("start", temps)
temps.sort()
print("sort ", temps)
new_temp = 27
temps.append(new_temp)
print("add  ", temps)
print("max  ", max(temps))
print("min  ", min(temps))
pre = [28, 24, 27]
pre.append(temps)
print("pre  ", pre)
aft = [31, 32, 30]
for temp in aft:
    temps.append(temp)
print("week2", temps)
print("avg  ", sum(temps) / len(temps))
```

Output

```
start [29, 26, 31, 28, 28, 30, 30]
sort [26, 28, 28, 29, 30, 30, 31]
add [26, 28, 28, 29, 30, 30, 31, 27]
max 31
min 26
pre [28, 24, 27, [26, 28, 28, 29, 30, 30, 31, 27]]
week2 [26, 28, 28, 29, 30, 30, 31, 27]
avg 28.625
```

$$\text{Average} = \frac{68+69+70+72+73+74+75+71+95+97+94}{11} = 78.0.$$

Discussion 2: What Is the Output?

- `pre.append(temps)` producted `[51, 53, 56, [68, 69, 70, 72, 73, 74, 75, 71]]`
- What if we wanted `[51, 53, 56, 68, 69, 70, 72, 73, 74, 75, 71]`?
- We saw how to do it with a for loop...is there another way?

Discussion 2: What Is the Output?

- `pre.append(temps)` producted `[51, 53, 56, [68, 69, 70, 72, 73, 74, 75, 71]]`
- What if we wanted `[51, 53, 56, 68, 69, 70, 72, 73, 74, 75, 71]`?
- We saw how to do it with a for loop...is there another way?

```
pre + temps  
pre.extend(temps)
```

Discussion 3: What Is the Output?

```
weekly_schedule = [  
    ["Meeting", "Emails", "Project Work", "Lunch",  
     "Code Review", "Call with Client", "Report Writing"],  
    ["Code", "Emails", "Design", "Lunch",  
     "Testing", "Meeting", "Documentation"],  
    ["Research", "Emails", "Development", "Lunch",  
     "Planning", "Team Meeting", "Presentation"],  
]  
  
l1 = weekly_schedule[0][:3]  
print(l1)  
l2 = weekly_schedule[1][-3:]  
print(l2)  
l3 = weekly_schedule[2][4]  
print(l3)
```

Output

Sublists or one item?

Discussion 3: What Is the Output?

```
weekly_schedule = [  
    ["Meeting", "Emails", "Project Work", "Lunch",  
     "Code Review", "Call with Client", "Report Writing"],  
    ["Code", "Emails", "Design", "Lunch",  
     "Testing", "Meeting", "Documentation"],  
    ["Research", "Emails", "Development", "Lunch",  
     "Planning", "Team Meeting", "Presentation"],  
]  
  
l1 = weekly_schedule[0][:3]  
print(l1)  
l2 = weekly_schedule[1][-3:]  
print(l2)  
l3 = weekly_schedule[2][4]  
print(l3)
```

Output

Work through line by line.

Discussion 3: What Is the Output?

```
weekly_schedule = [  
    ["Meeting", "Emails", "Project Work", "Lunch",  
     "Code Review", "Call with Client", "Report Writing"],  
    ["Code", "Emails", "Design", "Lunch",  
     "Testing", "Meeting", "Documentation"],  
    ["Research", "Emails", "Development", "Lunch",  
     "Planning", "Team Meeting", "Presentation"],  
]  
  
l1 = weekly_schedule[0][:3]  
print(l1)  
l2 = weekly_schedule[1][-3:]  
print(l2)  
l3 = weekly_schedule[2][4]  
print(l3)
```

Output

```
['Meeting', 'Emails', 'Project  
Work']
```

`[:3]` returns the first three elements of the first sublist.

Discussion 3: What Is the Output?

```
weekly_schedule = [  
    ["Meeting", "Emails", "Project Work", "Lunch",  
     "Code Review", "Call with Client", "Report Writing"],  
    ["Code", "Emails", "Design", "Lunch",  
     "Testing", "Meeting", "Documentation"],  
    ["Research", "Emails", "Development", "Lunch",  
     "Planning", "Team Meeting", "Presentation"],  
]  
  
l1 = weekly_schedule[0][:3]  
print(l1)  
l2 = weekly_schedule[1][-3:]  
print(l2)  
l3 = weekly_schedule[2][4]  
print(l3)
```

Output

```
['Meeting', 'Emails', 'Project  
Work']  
['Testing', 'Meeting',  
'Documentation']
```

`[-3:]` returns the last three elements of the second sublist.

Discussion 3: What Is the Output?

```
weekly_schedule = [  
    ["Meeting", "Emails", "Project Work", "Lunch",  
     "Code Review", "Call with Client", "Report Writing"],  
    ["Code", "Emails", "Design", "Lunch",  
     "Testing", "Meeting", "Documentation"],  
    ["Research", "Emails", "Development", "Lunch",  
     "Planning", "Team Meeting", "Presentation"],  
]  
  
l1 = weekly_schedule[0][:3]  
print(l1)  
l2 = weekly_schedule[1][-3:]  
print(l2)  
l3 = weekly_schedule[2][4]  
print(l3)
```

Output

```
['Meeting', 'Emails', 'Project  
Work']  
['Testing', 'Meeting',  
'Documentation']  
Planning
```

[4] picks one element from the third sublist, so the result is a string, not a list.

Discussion 4: Squares of Numbers Divisible by Three

Goal: in as few lines as possible, create a list of all square numbers x^2 with $1 \leq x \leq 100$ that are divisible by 3.

- All numbers between 1 and 100
- Multiples of three
- Squares of all numbers between 1 and 100
- Put it together

Discussion 4: Start with All Numbers

```
numbers = [number for number in range(1, 101)]
```

Discussion 4: Keep Only Multiples of 3

```
multiples_of_three = [number for number in range(1, 101)  
                        if number % 3 == 0]
```

Discussion 4: Square Every Number

```
squares = [number ** 2 for number in range(1, 101)]
```

Discussion 4: Put It Together

```
square_numbers = [number ** 2 for number in range(1, 101)  
                  if number % 3 == 0]
```

Discussion 5: Book Database Setup

```
library = {  
  "978-1786074126": {"title": "How We Disappeared",  
                    "author": "Jing-Jing Lee",  
                    "year": 2019, "genre": "Historical Fiction"},  
  "978-1509855315": {"title": "Ponti",  
                    "author": "Sharlene Teo",  
                    "year": 2018, "genre": "Fiction"},  
  "978-0063161603": {"title": "Now You See Us",  
                    "author": "Balli Kaur Jaswal",  
                    "year": 2023, "genre": "Mystery"}  
}
```

Aside on ISBN numbers

- First three digits makes it a valid bar code: 978
- Next digits encode language / location
- Next digits encode publisher
- Whatever is left (except for last digit) identifies book
- Last digit is checksum

Aside on ISBN numbers

- First three digits makes it a valid bar code: 978
- Next digits encode language / location
 - ▶ 978-**0/1**: English language
 - ▶ 978-**2**: French language
 - ▶ 978-**3**: German language
 - ▶ 978-**4**: Japan (nation)
 - ▶ 978-**5**: CIS (group of nations)
 - ▶ 978-**65**: Brazil (nation)
 - ▶ 978-**7**: PRC (nation)
 - ▶ 978-**981**: Singapore (nation)
- Next digits encode publisher
- Whatever is left (except for last digit) identifies book
- Last digit is checksum

Aside on ISBN numbers

- First three digits makes it a valid bar code: 978
- Next digits encode language / location
- Next digits encode publisher
 - ▶ 978-1**01**: Penguin
 - ▶ 978-1**4012**: DC Comics
- Whatever is left (except for last digit) identifies book
- Last digit is checksum

Aside on ISBN numbers

- First three digits makes it a valid bar code: 978
- Next digits encode language / location
- Next digits encode publisher
- Whatever is left (except for last digit) identifies book
- Last digit is checksum

```
total = 0
for idx, digit in enumerate(int(x) for x in "945050147621"):
    # Multiply by 1 for even indices (0, 2, ...), 3 for odd indices (1, 3, ...)
    total += digit * (1 + 2 * (idx % 2))
    print(total, digit, 1 + 2 * (idx % 2))

check_digit = (10 - total % 10) % 10 # Mod 10 again in case 0
print(check_digit)
```

Aside on ISBN numbers

- First three digits makes it a valid bar code: 978
- Next digits encode language / location
- Next digits encode publisher
- Whatever is left (except for last digit) identifies book
- Last digit is checksum

```
total = 0
for idx, digit in enumerate(int(x) for x in "945050147621"):
    # Multiply by 1 for even indices (0, 2, ...), 3 for odd indices (1, 3, ...)
    total += digit * (1 + 2 * (idx % 2))
    print(total, digit, 1 + 2 * (idx % 2))

check_digit = (10 - total % 10) % 10 # Mod 10 again in case 0
print(check_digit)
```

- Similar to how Unicode characters are determined

Discussion 5A: Print the Whole Database

Write code to print each ISBN together with its title.

Discussion 5A: Print the Whole Database

```
print("Library collection:")  
for isbn, book in library.items():  
    print(isbn, book["title"])
```

Output:

```
Library collection:  
978-1786074126 How We Disappeared  
978-1509855315 Ponti  
978-0063161603 Now You See Us
```

Discussion 5B: Look Up One ISBN

Look up ISBN from user `input`, retrieve the book (think about error cases), and print all the fields of the book information.

Discussion 5B: Look Up One ISBN

```
isbn_to_lookup = input("Book ISBN:")  
book_info = library.get(isbn_to_lookup, {})  
for field in book_info:  
    print(field, book_info[field])
```

Output:

```
title Now You See Us  
author Balli Kaur Jaswal  
year 2023  
genre Mystery
```

Discussion 5C: Add a New Book

Add *Raffles Hotel* by Ryu Murakami, published in 1989, in the genre Mystery.

Discussion 5C: Add a New Book

```
new_book_isbn = "978-4087498288"
library[new_book_isbn] = {
    "title": "Raffuruzu Hoteru",
    "author": "Ryu Murakami",
    "year": 1989,
    "genre": "Mystery"
}
print("added", library[new_book_isbn]["title"])
print(library[new_book_isbn])
print(library.keys())
```

Output:

```
added Raffuruzu Hoteru
{'title': 'Raffuruzu Hoteru', 'author': 'Ryu Murakami', 'year': 1989, 'genre': 'Mystery'}
dict_keys(['978-1786074126', '978-1509855315', '978-0063161603', '978-4087498288'])
```

Discussion 5D: Remove a Book

Remove ISBN 978-1509842794 from the database and print the removed entry.

Discussion 5D: Remove a Book

```
for isbn_to_remove in ["978-1509842794", "978-1509855315"]:  
    removed_book = library.pop(isbn_to_remove, "Book not found")  
    if isinstance(removed_book, dict):  
        print("removed", removed_book["title"])  
    else:  
        print(removed_book)  
print(library.keys())
```

Output:

```
removed Ponti
```

Discussion 5E: Update a Field

Update the title for *Raffuruzu Hoteru* to Raffles Hotel, then print the updated entry.

Discussion 5E: Update a Field

```
isbn_to_update = "978-4087498288"  
library[isbn_to_update]["title"] = "Raffles Hotel"  
print(library[isbn_to_update]["title"])  
print(library[isbn_to_update])
```

Output:

```
{'title': 'Raffles Hotel', 'author': 'Ryu Murakami', 'year': 1989, 'genre': 'Mystery'}
```

Discussion 5F: Search by Author

Find all books by Jing-Jing Lee.

Discussion 5F: Search by Author

```
author_to_search = "Jing-Jing Lee"
books_by_author = [book for book in library.values()
                    if book["author"] == author_to_search]
for book in books_by_author:
    print(book["title"])
```

Output:

```
How We Disappeared
```

Discussion 5G: Filter by Genre

Create a new dictionary containing only the books in the genre `Mystery`.

Discussion 5G: Filter by Genre

```
genre_to_filter = "Mystery"
books_in_genre = {isbn: book for isbn, book in library.items()
                  if book["genre"] == genre_to_filter}
for book in books_in_genre:
    print(isbn, book["title"])
```

Output:

```
978-0063161603 Now You See Us
978-4087498288 Raffuruzu Hoteru
```

Discussion 6: Palindromes in a Sentence

Given a sentence, output all words that are palindromes.

```
sentence = "level civic python radar banana madam code"
```

Discussion 6: Palindromes in a Sentence

```
sentence = "rotator civic python redder radar banana madam codec malayalam"  
palindromes = [word for word in sentence.split()  
                if word == word[::-1]]  
print(palindromes)
```

Discussion 6: Palindromes in a Sentence

```
sentence = "rotator civic python redder radar banana madam codec malayalam"  
palindromes = [word for word in sentence.split()  
                if word == word[::-1]]  
print(palindromes)
```

Other solutions:

- `"".join(reversed(word)) == word`
- `word[:ceil(len(word)//2)] == word[:len(word)//2-(len(word)+1)%2:-1]`

Discussion 7: B-Sprache

Translate each word into B-Sprache by doubling every vowel with a b in between.

Example: arbeit → abarbebeibit

Discussion 7: B-Sprache

```
sentence = "Franz jagt im komplett verwahrlosten Taxi quer durch Bayern"
cipher = sentence.lower()
for vowel in "aeiou":
    cipher = cipher.replace(vowel, vowel + "b" + vowel)

print(cipher)
```

```
cipher = ""
for char in sentence.lower():
    if char in "aeiou":
        cipher += char + "b" + char
    else:
        cipher += char
```

```
cipher = "".join("%sb%s" % x if x in "aeiou" else x for x in sentence.lower())
```

Output: frabanz jabagt ibim kobomplebett veberwabahrlobosteben tabaxibi
qubueber duburch babayebern

Takeaways

- String methods transform text.
- List methods can change a list in place or create nested structure.
- Slices return sublists, while indexing returns one element.
- List comprehensions combine generation, transformation, and filtering.
- Dictionaries work well for structured records (like json).
- String processing problems often combine loops, conditions, and membership tests.